



IBM BOB 2.0 HACKATHON · DEVELOPER ONBOARDING WORKFLOW

Understand any codebase before lunch.

Paste a public GitHub repository URL. Five analyst agents run in parallel against the real source and return an onboarding map: architecture, execution traces, house rules, risk radar, three shippable first tasks and a seven-step ramp-up plan.

Every claim cites the file it came from. No evidence, no claim.

Team solodev

OnboardPilot

Built with IBM Bob IDE

Onboarding is the most expensive workflow nobody measures

01

Days of archaeology

The README is a version behind. The diagram describes a system that no longer exists. Architecture lives in one senior engineer's head.

02

The same questions, forever

Where does a request enter? What owns this state? Why is this module like this? A senior engineer answers each one again for every new joiner.

03

Rework, not just delay

Early pull requests bounce because the newcomer touched the wrong layer or duplicated a utility that already existed three directories away.

And generic AI assistants make it worse first

Ask a chatbot how a codebase works and you get fluent, confident prose with no way to check it. A newcomer cannot tell a correct answer from an invented one — so they either trust it and break something, or ignore it and go back to reading.

A verified onboarding map, in under a minute

- **What is this system** — layered architecture with per-module responsibilities
- **How execution starts** — real entry points and an end-to-end trace
- **House rules** — the conventions that only exist in people's heads
- **Risk radar** — the landmines that cause rework, ranked by severity
- **Three first tasks** — real files, real steps, objective done-criteria
- **A seven-step ramp-up plan** — each step with a self-verifiable checkpoint

Results stream live: each section appears the moment its agent finishes.

TRACKED FILES
104

FILES READ
48

DOCS FOUND
1

TEST FILES
37

MANUAL RAMP-UP (EST.)
~21 h

ARCHITECTURE MAPPER

What this codebase is

Ky is a tiny, dependency-free HTTP client that wraps the WHATWG Fetch API with a friendlier surface: method shortcuts, retries, timeouts, hooks, base URLs and Standard Schema response validation. The public entry point source/index.ts builds a callable KyInstance via createInstance, delegating all real work to the Ky class in source/core/Ky.ts. That core class normalizes options, runs hooks, performs fetch with timeout and retry logic, and wraps responses in a ResponsePromise with body shortcuts. Errors, types and small pure utilities are split into their own directories.

TypeScriptWHATWG Fetch APINode.js >=22AVA (test runner)tsx xo (lint)Playwright (browser tests)Standard Schema

Entry / Public Interface

Defines the callable ky function, its instance methods and the exported error/type surface.

index

source/index.ts

Creates the ky instance, wires method shortcuts, create/extend and re-exports public API.

source/index.ts

types

source/types

Declares public option, hook, retry, response and Standard Schema types.

source/types/options.tssource/types/ky.tssource/types/hooks.ts

source/types/ResponsePromise.ts

Core Request Engine

Implements the actual request lifecycle: normalization, hooks, fetch, retry and response wrapping.

Ky class

source/core/Ky.ts

Central class that normalizes options, runs hooks, calls fetch, retries and wraps responses.

source/core/Ky.ts

constants

source/core/constants.ts

Runtime capability detection, method lists, option registries and retry markers.

source/core/constants.ts

retry-timing

source/core/retry-timing.ts

Parses Retry-After and rate-limit reset headers into retry delays.

source/core/retry-timing.ts

Errors and Type Guards

Defines the error hierarchy thrown across the request lifecycle and cross-realm guards for them.

errors

source/errors

One class per failure mode: HTTP, network, timeout, size, forced retry, schema, base KyError.

source/errors/KyError.tssource/errors/HTTPError.tssource/errors/NetworkError.tssource/errors/TimeoutError.ts

type-guards

source/utls/type-guards.ts

isKyError/isHTTPError/isNetworkError etc., handling cross-realm instanceof failures.

source/utls/type-guards.ts

Utilities / Integration

Pure helpers for merging options, body streaming, timeouts,

Paste a URL → read the map → commit ONBOARDING.md for the next person

onboardpilot

HOW IT WORKS

Five specialists, one shared context, zero invented file paths

01

Context assembly

Fetch the tree, drop dependencies and build output, rank what actually explains a codebase — docs, manifests, entry points, shallow core modules — and read it into one bounded snapshot.

02

Parallel fan-out

Five agents run concurrently over that same snapshot: Architecture Mapper, Entry-Point Detective, Convention & Risk Radar, First-Task Generator, Ramp-Up Planner.

03

Verification

Every statement carries the real file paths behind it, and every agent must declare what it could *not* determine. The report links each citation straight to GitHub.



OnboardPilot understand any codebase before lunch

Download ONBOARDING.md

JSON

IBM Bob 2.0 Hackathon · developer onboarding workflow

Understand any codebase before lunch.

Server-Sent Events · one event per agent · partial failure never aborts the run

9.8s · 5 agents

WHY IT IS DIFFERENT

Evidence, not vibes

testing

Add a compile-time test in test-d/ for any public type change, in addition to a runtime test in test/.

tsconfig.test.json

test-d/json-undefined-schema.ts

test/main.ts

FIRST-TASK GENERATOR

Your first three contributions

I picked tasks anchored in files whose contents I could actually read: the error classes, the type-guard module, the retry-timing parser, and the test/type-test layout. Each task is small, touches real source plus its matching test file, and is verifiable with the repo's own 'npm test' pipeline (xo, tsc, ava).

01

starter

Add a 'isSchemaValidationError' type guard and export it from 'source/index.ts'

Every other Ky error class has a guard in 'source/utils/type-guards.ts', but 'SchemaValidationError' is deliberately excluded from 'isKyError', leaving users no narrowing helper. It is a self-contained addition that teaches the error taxonomy and the cross-realm guard pattern.

source/utils/type-guards.ts

source/index.ts

source/errors/SchemaValidationError.ts

test/type-guards.ts

test-d/error-names.ts

STEPS

1.

Read 'isErrorType' and the existing guards in 'source/utils/type-guards.ts' to copy the exact pattern.

2.

Add 'isSchemaValidationError(error: unknown): error is SchemaValidationError' using 'isErrorType', importing the class from '..../errors/SchemaValidationError.js'.

3.

Export the new guard from the 'source/index.ts' export block that already lists 'isKyError', 'isHTTPError', etc.

4.

Add runtime assertions in 'test/type-guards.ts' covering a real 'SchemaValidationError' and a plain 'Error'.

5.

Add a type-narrowing assertion in 'test-d/error-names.ts' alongside the existing guard imports.

DONE WHEN

✓

'isSchemaValidationError' is exported from 'source/index.ts' and narrows to 'SchemaValidationError'.

✓

'npm test' passes, including 'xo' lint and the 'tsc --project tsconfig.test.json' type tests.

02

starter

Add unit tests for 'getRetryTimingHeader' date parsing in 'source/core/retry-timing.ts'

'source/core/retry-timing.ts' implements three HTTP-date formats plus a timestamp-vs-delay heuristic, and 'test/retry-timing.ts' already exists as the natural home. Writing tests against a pure function requires no network or browser setup, so a newcomer can iterate fast.

source/core/retry-timing.ts

test/retry-timing.ts

source/utils/normalize.ts

STEPS

1.

Read 'source/core/retry-timing.ts' to enumerate the IMF-fixdate, RFC 850 and asctime patterns and the 'timestampThreshold' branch.

2.

Open 'test/retry-timing.ts' and follow its existing import and assertion style.

3.

Add cases for a plain delay-seconds header, an IMF-fixdate header, and a malformed date that must fall back to 'undefined'.

4.

Add a case proving a numeric value below 'timestampThreshold' is treated as seconds, not an epoch timestamp.

5.

Run 'npx ava test/retry-timing.ts' to confirm only the new file runs.

DONE WHEN

✓

New assertions cover all three date formats and the delay-seconds path.

03

intermediate

Document and test the 'replaceOption' helper's interaction with 'ky.extend()'

'replaceOption' is exported from 'source/index.ts' and implemented in 'source/utils/merge.ts' via a private 'replaceSymbol', but its behavior is subtle: 'extend()' deep-merges by default and 'replaceOption' opts out. A contributor learns the merge pipeline while adding coverage for an under-tested public API.

source/utils/merge.ts

source/index.ts

test/merge.ts

test-d/normalized-options.ts

STEPS

1.

Read 'getReplaceState' and 'deepMergeInternal' in 'source/utils/merge.ts' to see how the 'replaceSymbol' marker is detected.

2.

Check whether 'test/merge.ts' exists in the tree; if not, create it following the 'ava' import style used in 'test/main.ts'.

3.

Add a test where a parent instance sets 'hooks.beforeRequest' and a child uses 'replaceOption' to replace rather than append.

4.

Add a contrasting test without 'replaceOption' asserting the default append/merge behavior.

5.

Add a type-level assertion in 'test-d/normalized-options.ts' if the helper's generic signature needs pinning.

DONE WHEN

✓

A test proves 'replaceOption' replaces the parent value while the default path merges.

✓

The new test file is picked up by the 'ava' config in 'test-d/normalized-options.ts'.

Cited, not asserted

Every module, risk and task carries file chips that open the exact file on GitHub, plus a full evidence index. Claim without evidence is not allowed by the prompt contract.

Admitted ignorance

Each analysis ships an "honest gaps" section. Knowing what to ask is half of onboarding.

Checkpoints, not reading lists

"Explain why the request passes through the middleware chain twice" — not "read src/core".

A reusable artifact

The whole report exports as ONBOARDING.md and can be committed next to the code.

Bob was the development partner, not a code autocomplete

Plan mode

Read the repository and the hackathon rules, then produced the hardening plan — files, risks and effort — that the rest of the build followed.

Agent mode

Multi-file changes under an approved plan: ranking heuristics, a named cap on test files in context, stronger grounding rules in the shared prompt.

Code reviews

Crafted repository identifiers, unbounded reads, missing timeouts, prompt injection from third-party source. Findings fixed, not filed.

Subagents & parallel tasks

Five prompt contracts red-teamed in isolated contexts, hunting for the input that would make an agent cite a file that does not exist.

Document understanding

The hackathon guide and submission rules became our compliance checklist; Bob then audited the README against the actual source.

Security audit

A security-auditor mode checked credential handling, untrusted repository text, and SSRF through user input.

Next.js 16 · React 19 · TypeScript · DeepSeek · Vercel

lib/github.ts

Repository tree fetch, filtering, onboarding-value ranking, adaptive per-file budget, parallel raw reads with timeouts.

lib/agents.ts

One shared system prompt built once and reused by all five agents, so DeepSeek's prefix cache keeps the fan-out fast and cheap.

lib/deepseek.ts

JSON-mode calls with retry, truncation detection and a repair pass that salvages a cut-off object instead of losing the agent.

app/api/analyze

Server-Sent Events route. Per-agent isolation: one failure never aborts the other four, and the UI offers a one-click retry.

components/

Live agent board, evidence-linked report, streaming results as each agent lands.

Verification

Smoke test for the context layer that spends zero model tokens, plus unit tests for the pure helpers.

Next.js 16

React 19

TypeScript

Tailwind v4

DeepSeek

GitHub API

Server-Sent Events

Vercel

IMPACT

Under a minute, versus hours to days of manual archaeology

< 60s

full onboarding map for a mid-size repository

5

specialist agents over one shared context

100%

of claims carry a file citation

0

invented file paths permitted by the prompt contract

What it does not do

It does not replace the conversation with the team — it removes the reading work *before* that conversation, so the conversation starts at a much higher level. That is exactly what the "questions only your team can answer" section exists for.



TRY IT

Paste a repository.
Get a map you can verify.

Live demo: <https://solodev-dun.vercel.app>

Source: github.com/Careylq/solodev

Open source under the MIT License · IBM Bob task session summaries in [bob_sessions/](#)